

LABORATORY OBSERVATION

Course: Full Stack Web Development
Course Code: 23CM4121
Experiment No.: 3
Student Name: Hansikha
Roll No.: A24126552264
Date: 12-09-2026

1. TITLE

Implement a Dynamic Task Manager Using HTML, CSS and JavaScript

2. AIM

To develop a dynamic task-management webpage using HTML, CSS and JavaScript that allows users to add tasks, mark tasks as completed and remove tasks through DOM manipulation and event handling

3. OBJECTIVE

- To create a simple task-management interface using HTML.
 - To apply CSS styling to the task manager.
 - To access webpage elements using JavaScript DOM methods.
 - To add tasks dynamically based on user input.
 - To validate empty task input.
 - To implement task completion functionality, remove tasks dynamically.
 - To display an appropriate message when no tasks are available.
-

4. THEORY

A dynamic webpage can respond to user actions by using JavaScript event listeners and DOM manipulation. The Document Object Model represents HTML elements as objects that can be accessed and modified using JavaScript. In this task manager application, the HTML document contains a text box, an Add button, a message area and a task container. CSS is used to create the layout and visual appearance of the application. JavaScript selects these elements using `document.getElementById()`. When the user clicks the Add button, the entered task is trimmed and checked. If the input is empty, an alert is displayed. Otherwise, a new task element is created dynamically. Each task contains a task name, a Complete button and a Delete button. The Complete button adds or removes the `'finished'` CSS class, which changes the appearance of the task. The Delete button removes the task from the webpage. When all tasks are deleted, the application displays the **No tasks available** message again.

The working sequence is:

Enter task → Validate input → Create task → Add buttons → Complete/Delete task → Update interface

5. ALGORITHM / PROCEDURE

1. Create the HTML structure for the task manager.
2. Add a text input field for entering a task.
3. Add an Add button.
4. Create a message area for displaying the empty-list status.
5. Create a container for dynamically generated tasks.
6. Link the CSS and JavaScript files.
7. Select the required elements using `getElementById()`.

8. Attach a click event to the Add button.
 9. Read and trim the entered task value.
 10. Check whether the input is empty.
 11. Display an alert when no task is entered.
 12. Create a task container dynamically for valid input.
 13. Create the task name and action buttons.
 14. Add a Complete event to change the task status.
 15. Add a Delete event to remove the task.
 16. Append the task elements to the task container.
 17. Hide the empty-list message when a task is added.
 18. Display the empty-list message when all tasks are removed.
 19. Clear the input field after adding a task.
 20. Test the application with different task operations.
-

6. PROGRAM

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Task Manager</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

  <div class="todo-container">

    <h1>Task Manager</h1>

    <div class="task-input">
      <input type="text" id="newTask" placeholder="Enter your task...">
      <button id="addButton">Add</button>
    </div>

    <div id="message">No tasks available.</div>

    <div id="tasks"></div>

  </div>

  <script src="script.js"></script>

</body>
</html>
```

script.js

```
const inputBox = document.getElementById("newTask");
const addButton = document.getElementById("addButton");
const taskContainer = document.getElementById("tasks");
const message = document.getElementById("message");

addButton.addEventListener("click", addNewTask);

function addNewTask() {

  let taskValue = inputBox.value.trim();

  if (taskValue === "") {
    alert("Please enter a task!");
    return;
  }

  // Create task container
  const task = document.createElement("div");
  task.className = "task";

  // Create task name
  const taskName = document.createElement("span");
  taskName.className = "task-name";
  taskName.innerText = taskValue;

  // Create buttons container
  const buttonArea = document.createElement("div");
  buttonArea.className = "task-buttons";

  // Create complete button
```

```

const completeButton = document.createElement("button");
completeButton.innerText = "Complete";
completeButton.className = "done";

// Create delete button
const deleteButton = document.createElement("button");
deleteButton.innerText = "Delete";
deleteButton.className = "remove";

// Complete button functionality
completeButton.addEventListener("click", function () {

    taskName.classList.toggle("finished");

    if (taskName.classList.contains("finished")) {
        completeButton.innerText = "Completed";
    } else {
        completeButton.innerText = "Complete";
    }

});

// Delete button functionality
deleteButton.addEventListener("click", function () {

    task.remove();

    if (taskContainer.children.length === 0) {
        message.style.display = "block";
    }

});

// Add buttons to button area
buttonArea.appendChild(completeButton);
buttonArea.appendChild(deleteButton);

// Add task name and buttons to task
task.appendChild(taskName);
task.appendChild(buttonArea);

// Add task to webpage
taskContainer.appendChild(task);

// Hide empty message
message.style.display = "none";

// Clear input
inputBox.value = "";

}
// Clear input box
taskInput.value = "";

```

style.css

```

* {
    box-sizing: border-box;
}

body {
    margin: 0;
    font-family: Arial, sans-serif;
    background: #e8f0fe;
}

.todo-container {
    width: 550px;
    margin: 70px auto;
    padding: 30px;
    background: white;
    border-radius: 12px;
    box-shadow: 0 5px 20px rgba(0, 0, 0, 0.15);
}

h1 {
    text-align: center;
    margin-bottom: 25px;
}

.task-input {
    display: flex;
    gap: 8px;
}

#newTask {
    flex: 1;
    padding: 12px;
    font-size: 16px;
    border: 1px solid #aaa;
    border-radius: 5px;
}

#addButton {
    padding: 12px 20px;
    border: none;
    border-radius: 5px;
    background: #673ab7;
    color: white;
}

```

```

        cursor: pointer;
    }

    #addButton:hover {
        background: #512da8;
    }

    #message {
        text-align: center;
        margin: 25px 0;
        color: #777;
    }

    .task {
        display: flex;
        align-items: center;
        justify-content: space-between;
        padding: 12px;
        margin: 10px 0;
        background: #f5f5f5;
        border-left: 4px solid #673ab7;
        border-radius: 5px;
    }

    .task-name {
        flex: 1;
        font-size: 16px;
    }

    .task-buttons button {
        margin-left: 6px;
        padding: 7px 12px;
        border: none;
        border-radius: 4px;
        cursor: pointer;
    }

    .done {
        background: #4caf50;
        color: white;
    }

    .remove {
        background: #f44336;
        color: white;
    }

    .finished {
        text-decoration: line-through;
        color: #888;
    }

    flex: 1;
    font-size: 16px;
}

.completed {
    text-decoration: line-through;
    color: gray;
}

.task-buttons {
    display: flex;
    gap: 5px;
}

.complete-btn {
    background-color: #2196F3;
    color: white;
    border: none;
    padding: 7px 10px;
    cursor: pointer;
    border-radius: 4px;
}

.delete-btn {
    background-color: #f44336;
    color: white;
    border: none;
    padding: 7px 10px;
    cursor: pointer;
    border-radius: 4px;
}

```

7. CODE EXPLANATION

index.html – Defines the page structure, including the My To-Do List heading, task input, Add Task button, empty message, task list and links to CSS and JavaScript.

script.js – Uses DOM selection and click event listeners. It validates input, dynamically creates task elements, toggles completion state, deletes tasks, controls the empty message and clears the input.

style.css – Styles the page background and central container, input area, Add Task button, task rows, Complete/Delete buttons and the completed-task line-through effect.

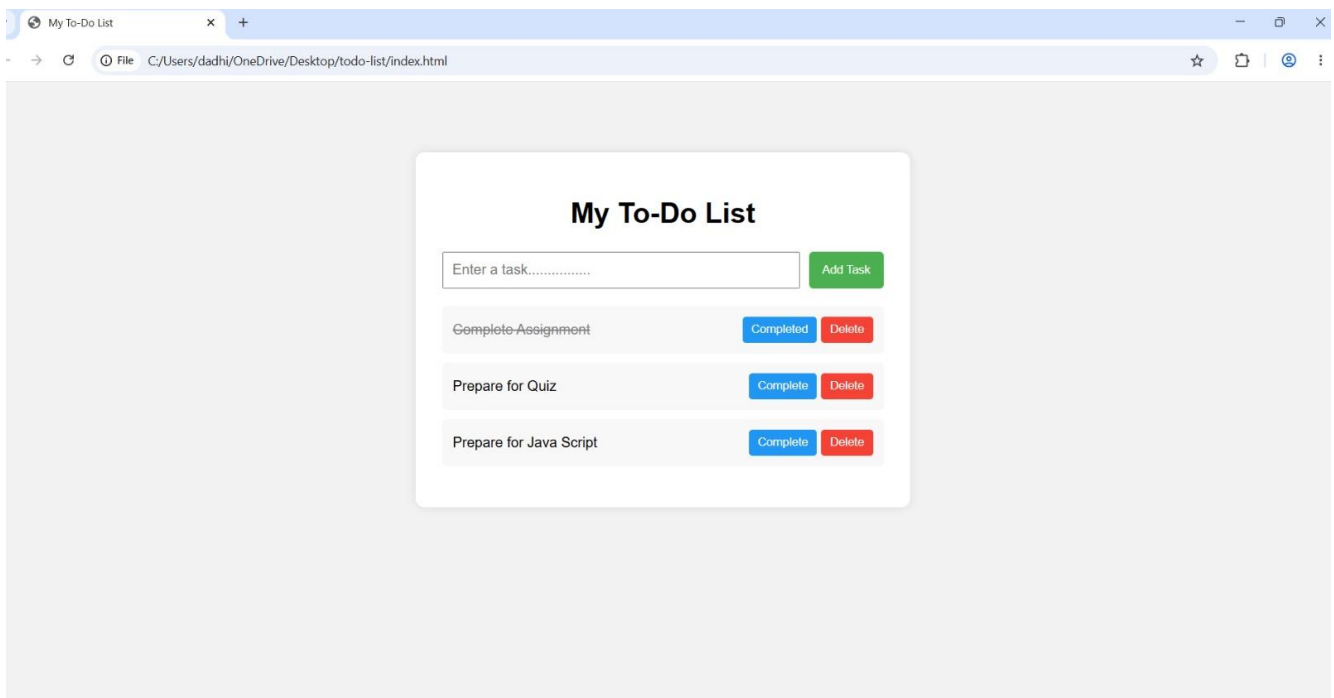
8. TEST CASES AND EXECUTION DATA

The following cases are based on the implemented event handlers and the supplied repository output screenshot. Where a behavior is not directly visible in the screenshot, it is identified as code-supported rather than claimed as an observed execution result.

Test Case	Input / Action	Expected Result	Actual / Evidence	Status
TC-01	Enter a task and click Add Task	Task should be added to the list.	Tasks are visible in the supplied output screenshot.	Pass
TC-02	Click Complete for a task	Task text should be crossed out and button should change to Completed.	Completed task with strikethrough and “Completed” button is visible in screenshot.	Pass
TC-03	Click Delete for a task	Task should be removed; empty message should appear when none remain.	Delete behavior is implemented in script.js; no deletion state is shown in the supplied screenshot.	Code verified
TC-04	Click Add Task with empty input	Alert “Please enter a task.” should be displayed.	Empty-input validation is implemented in script.js; alert state is not shown in screenshot.	Code verified

9. OUTPUT

Output 1: To-Do List application showing added tasks, one completed task and active task.



10. RESULT

Thus, a dynamic Task Manager application was successfully developed using HTML, CSS and JavaScript. The application allows users to add tasks, change their completion status and delete tasks. DOM manipulation and event listeners were used to update the webpage dynamically according to user actions.